

Introduction

This lab sheet supports your learning for the first “half” of the module (i.e. the content that we go through until the end of W04). All of these exercises are *formative*: this means that you can work on them by yourself, with a partner, or even in groups. There are no grades available for these, and you are encouraged to discuss your solution with the other students, or, if you continue to have doubts/questions, with a teaching assistant and/or a lecturer.

Types of exercises. There are a number of different tasks that develop different skills. Some tasks require you to do something with “paper and pencil”, some tasks require perhaps a bit of further reading/research, and some require to implement something in Python.

Python

If you are experienced with Python then use whatever environment you like to work with; just ensure that when it comes to the coursework submission, your code happily runs with Python 3.

If you are not so familiar with Python, then I recommend to use PyCharm Community by JetBrains: <https://www.jetbrains.com/pycharm/> (free to use, the commercial version is also free for students but you must provide proof of student status). The main reason for this recommendation is that it offers a reasonably easy to use IDE, and you can set up virtual environments for different projects. If you do so, then, if you mess up one VE with libraries etc., you only mess up one project.

1 One-time encryption

Consider a one-time encryption scheme called Enc_1 where messages are bit-strings, and each bit is sampled according to the following distribution $\Pr[m = 0] = 1/4$ and $\Pr[m =$

1] = 3/4. Keys are also bit-strings, sampled freshly for each message, and of equal length to a message, with the following distribution $\Pr[k = 0] = 1/2$ and $\Pr[k = 1] = 1/2$.

Consider another one-time encryption scheme called Enc_2 where messages are bit-strings, and each bit is sampled according to the following distribution $\Pr[m = 0] = 1/2$ and $\Pr[m = 1] = 1/2$. Keys are also bit-strings, sampled freshly for each message, and of equal length to a message, with the following distribution $\Pr[k = 0] = 1/4$ and $\Pr[k = 1] = 3/4$.

Consider yet another one-time encryption scheme called Enc_3 where messages are bit-strings, and each bit is sampled according to the following distribution $\Pr[m = 0] = 1/3$ and $\Pr[m = 1] = 2/3$. Keys are also bit-strings, sampled freshly for each message, and of equal length to a message, with the following distribution $\Pr[k = 0] = 1/4$ and $\Pr[k = 1] = 3/4$.

- a. For each of the schemes sample a 12 bit message from the respective message space adhering to the given distribution. Then sample a corresponding key string adhering to the given distribution, and encrypt the message using the key. What do you notice about the distribution of the bits in the ciphertext for each scheme?
- b. Work out, using the given distributions, what is $\Pr[M = m | C = c]$.
- c. What would adherence to Kerckhoff's principle (regarding the no security by obscurity) imply in relation to the three constructions?
- d. Are any of the encryption schemes secure and why? Can you prove perfect secrecy for any/some/all of them according to Shannon's definition?

2 The operation $x \oplus c$ is a bijection

We used a key concept to argue about the security of the one-time pad, and in fact any construction that relies on a Boolean addition. This key concept is that the Boolean addition $x \oplus c$, with $x, c \in \{0, 1\}^2$ and c being a constant, is a bijection for all choices of c .

We also used a property of bijections, which is that if they go from a pre-image space $\mathcal{X}$, with a uniform probability distribution $p_{\mathcal{X}}$, to an image space $\mathcal{Y}$, then the distribution $p_{\mathcal{Y}}$ must also be uniform.

What is a bijection? (Wikipedia is your friend). Why does a bijection preserve the uniform distribution distribution, i.e. if the elements in $\mathcal{X}$ occur with a uniform distribution, then the elements in $\mathcal{Y}$ also have a uniform distribution? Consider the simplest case where $\mathcal{X} = \{0, 1\}$, and c is thus also just a single bit.

3 One-time pad with other operations

The one-time pad is defined via exclusive-oring the message with the key (in encryption) and via exclusive-oring the ciphertext with the key (in decryption). What would happen if we would use another Boolean operation instead, e.g. the bit-wise AND or the bit-wise OR?

Write for both these options the full construction (Key Generation, Encryption, Decryption) as a semi-formal algorithm. What can you say about correctness and security?

4 One-time pad without the all-zero key

One-time pad encryption with the all zero key results in the ciphertext being equal to the message. This doesn't look very secure. As an alternative, a construction is proposed where keys are still sampled uniformly at random from the key space, but the all zero key is always discarded (and instead another random key is sampled). What can you say about the security of this alternative construction?

5 One-time real-random security implies one-time left-right security

Show that the following claim, made in the lecture (in passing), is true.

Consider the two general Real and Random oracles for one-time symmetric encryption (see picture below): the claim is that if a cryptosystem is real-random secure (i.e. the outputs from the real oracle cannot be distinguished from outputs of the random oracle), then, it is also left-right secure.

6 Distrusting organisations

Consider the situation where MI5 and MI6 want to share the secret combination to a vault. They want this to be such that five employees of MI5 and six employees of MI6 together can open the vault. So only if eleven employees from both organisations come together (five from MI5 and six from MI6), should they be able to reveal the secret combination. How would you deal with this situation? Who would be creating the shares, and how would share distribution perhaps work in practice?

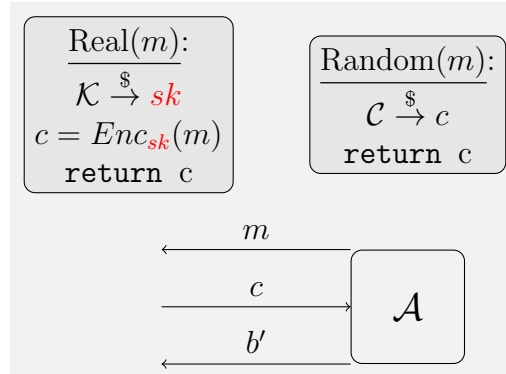


Figure 1: Exp RR-Pas. The real or random world are chosen by flipping a bit b . The adversary wins if $b' = b$.

7 Public recombination vector

We have parties $P_1, \dots, P_5$ who shared a secret s via a $(2, 5)$ Shamir secret sharing scheme over $\mathbb{Z}_{101}$. Compute the public recombination vector for the subset $\{P_1, P_2\}$ and for the subset $\{P_3, P_5\}$. Does the publication of these vectors help P_4 ?

8 Implement Shamir's scheme in Python

I provided a few helper functions so that you can implement a toy version of Shamir's secret sharing in Python. The script that I provided does run and it produces a random polynomial.

A polynomial is defined/represented as a list of coefficients a_i : the length of the list equals the degree of the polynomial minus one: $\{a_0, a_1, \dots, a_{d-1}\}$ is a list with d elements representing a polynomial of degree $d - 1$.

The functions that I provide are:

evaluate_polynomial(): takes three inputs; a list of coefficients representing the polynomial $f()$, an integer value x at which the polynomial is meant to be evaluated, and a modulus m . The function returns $f(x) \pmod{m}$.

modular_inverse(): takes two inputs; a value a that is meant to be inverted, and a modulus m . The function returns $a^{-1} \pmod{m}$.

generate_polynomial(): takes three inputs; a value for a_0 , the degree d of the polynomial, and a modulus. The function returns a list of coefficients : $\{a_0, a_1, \dots, a_d\}$ with $a_i \in \{0, \dots, m - 1\}$.

`lagrange_delta_i()`: takes three inputs; a set of values (x_i) , with $x_i \in \{1, 2, \dots\}$, and a point x at which the interpolation polynomial is evaluated. The function returns as many $\delta_i(x)$ as there were coordinates supplied at the respective i .

With these helper functions you should write the code for the empty functions `lagrange_evaluate`, `Stn_share` and `Stn_reconstruct`.

The function `lagrange_evaluate` should be able to return the result of evaluating a polynomial at x that was reconstructed using the Lagrange method. You should use the helper function `lagrange_delta_i` for this.

The function `Stn_share` and `Stn_reconstruct` implement the share and reconstruct functionality of a (t, n) Shamir secret sharing scheme.

9 Lost share

You set up a $(2, 30)$ Shamir secret sharing scheme over $\mathbb{Z}_{101}$. You gave out shares to David $(1, 13)$ and Tom $(3, 12)$, and also to Mark who is the second user in this system. Sadly he lost his share. Can David and Tom help him recover it?

10 Check out PyCryptoDome's Implementation

There are several Open Source implementations for Shamir's secret sharing available. One of them is part of the PyCryptoDome library (which is no deprecated): <https://pycryptodome.readthedocs.io/en/latest/src/protocol/ss.html>.

Install PyCryptoDome and use it to secret share the string "lovecryptography" in any form you fancy, and then reconstruct, all whilst using the library functions.

Try to make sense of the necessity to use an irreducible polynomial; how does this relate to how we defined Shamir's scheme?

How clear do you find their description of how Shamir's secret sharing scheme works?

11 MPC via Secret Sharing

We consider again a situation with three players, and threshold $t = 2$. Two secret values a and b are shared out among all players using some secret degree 1 polynomials over $\mathbb{Z}_{13}$. They use their shares to compute shares of $a \cdot b$, and then come together to reconstruct the product.

Value	P_1	P_2	P_3
a	5	8	11
b	7	9	11
$a \cdot b$	c_1	c_2	c_3

What are their shares c_i for the product initially? Verify that they hold a correct secret sharing of the 10 via the public recombination vector.

Then perform a degree reduction by assuming that each party uses the following secret polynomials:

$$P_1 : c_1 + 2x$$

$$P_2 : c_2 + 5x$$

$$P_3 : c_3 + 1x.$$

Work out the shares that all P_i produce and make clear which share is for which party. Then assume that the shares are sent out. Write down which shares the parties hold after they receive shares from the other parties. Next recombine the shares to derive c_i after the degree reduction. With the new shares for the product, show, with the public recombination vector, that they give the correct value of the product.

Finally, consider what happens if P_2 is taken over by a malicious adversary who changes the value of their share to 9 in the last step.

12 Only a billionaires' problem

Elon, Bill, Mark, and Jeff have an argument. Each of them received secret donations to their various humanitarian enterprises. Elon got 1 USD, Bill got 10 USD, Marc received 7 USD and Jeff got 3 USD. They do not want to reveal the exact amounts to each other, but they want to know who received the biggest donation, and they would also like to know the sum of all donations.

They have heard about the concept of secure multiparty computation and enlist you as a consultant. They have the following questions that they would like you to answer:

1. What solutions for secure multiparty computation are available at the moment?
2. Explain how one can quite easily compute the sum over their donations by using a secret sharing scheme that is linear in the addition over the integers.

3. Explain if/how they could set up a (t, n) Shamir secret sharing based protocol that computes the maximum over their secret inputs. This requires you to explain how the maximum can be efficiently computed via a circuit over $\mathbb{Z}_p$.